

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: Болотин А.Н.
Группа: М8О-301Бв-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: изучение основ 3D-графики — построения простых 3D-объектов, проекции на 2D-плоскость, а также работы с матрицами перспективной и ортографической проекции и аффинными преобразованиями.

Вариант задания: №3 — параллелепипед.

Требования к программе:

- использование Vulkan, GLFW и ImGUI;
- работа в реальном времени;
- динамическая смена проекции и трансформаций объекта через интерфейс пользователя.

Что реализовано: программа открывает окно, отрисовывает трёхмерный параллелепипед с цветными вершинами, предоставляет ImGUI-панель с переключателем проекции (перспективная/ортографическая) и слайдерами трансформаций (позиция, поворот, масштаб).

Метод решения

Математическая модель.

Для проецирования трёхмерной сцены на экран используется композиция из трёх матриц:

$$\mathbf{MVP} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M}$$

- **M (Model)** — матрица трансформации объекта. Задаётся через композицию смещения, поворотов вокруг трёх осей и масштабирования.
- **V (View)** — матрица камеры. Строится через `lookAt(eye, target, up)` — позиция камеры, точка, куда она смотрит, и вектор «вверх».
- **P (Projection)** — матрица проекции. В программе используются два типа:
 - **перспективная** (`glm::perspective`): дальние объекты визуально меньше, реализуется через деление на w-компоненту;
 - **ортографическая** (`glm::ortho`): размер объекта не зависит от расстояния до камеры.

Матрицы применяются к вершине справа налево: сначала M переводит вершину из локальных координат в мировые, затем V — в систему координат камеры, затем P — в клип-пространство. Финальное преобразование в пиксели экрана выполняет GPU через viewport.

Однородные координаты. Для задания точек и векторов используется четыре компоненты: $(x, y, z, 1)$ для точки, $(x, y, z, 0)$ для вектора. Это позволяет задать сдвиг через матричное умножение. После умножения на проекционную матрицу выполняется перспективное деление $(x/w, y/w, z/w)$ — именно оно даёт эффект уменьшения дальних объектов.

Геометрическая модель.

Параллелепипед задаётся 8 вершинами с размерами $1.6 \times 1.0 \times 0.7$. Каждая вершина содержит:

- позицию — три float-координаты (x, y, z) ;
- цвет — три float-компоненты (r, g, b) .

Так как GPU умеет растеризовать только треугольники, каждая грань параллелепипеда разбивается на 2 треугольника. Всего $6 \text{ граней} \times 2 \times 3 =$

36 индексов.

Для экономии памяти используется **индексный буфер** — каждая вершина хранится один раз, а треугольники ссылаются на неё по номеру. Без индексов пришлось бы дублировать вершины (36 вершин вместо 8).

Использованные объекты Vulkan.

Программа использует следующие объекты:

- **VkInstance, VkPhysicalDevice, VkDevice** — базовые объекты для работы с Vulkan.
- **VkSwapchainKHR** — цепочка изображений, которая отображается на экране.
- **VkRenderPass, VkFramebuffer** — описание прохода отрисовки и привязка к конкретному изображению swapchain.
- **VkBuffer + VmaAllocation** — три буфера данных:
 - вершинный (VK_BUFFER_USAGE_VERTEX_BUFFER_BIT) — массив вершин;
 - индексный (VK_BUFFER_USAGE_INDEX_BUFFER_BIT) — массив индексов;
 - uniform (VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT) — матрица MVP.

Память выделяется через библиотеку VMA.

- **VkDescriptorSetLayout, VkDescriptorPool, VkDescriptorSet** — механизм связывания uniform-буфера с шейдером. Шейдер читает матрицу по адресу set=0, binding=0.
- **VkPipelineLayout, VkPipeline** — описание графического конвейера: какие шейдеры использовать, как читать вершины, как растеризовать, как тестировать глубину.
- **VkShaderModule** — загруженные шейдеры (вершинный и фрагментный), скомпилированные в SPIR-V из GLSL.
- **VkCommandBuffer** — буфер команд, куда записывается последовательность действий для GPU.

Результаты

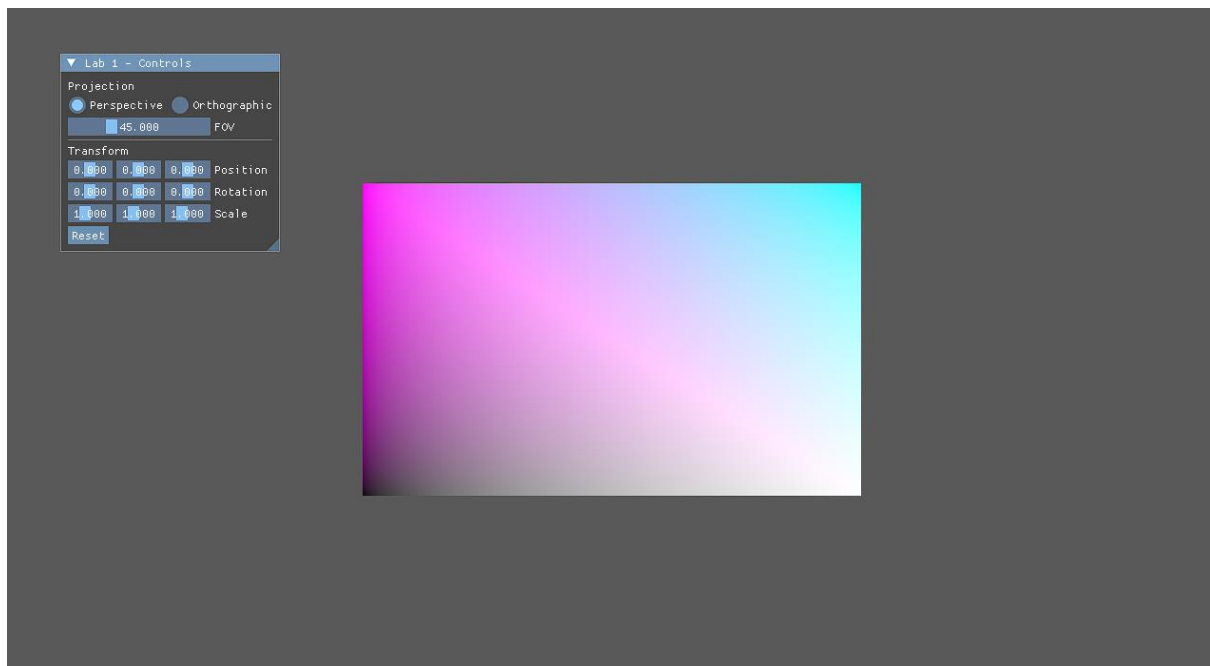
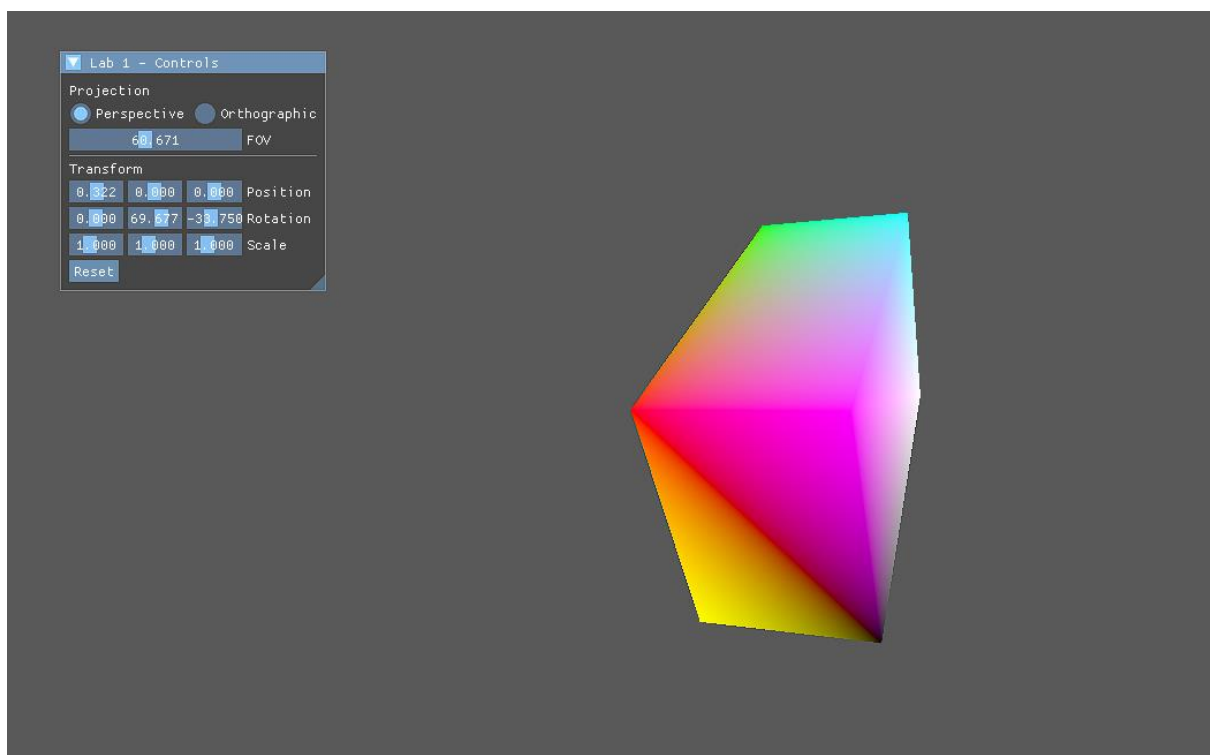


Рис. 1. Общий вид окна приложения: параллелепипед, отрисованный в перспективной проекции, и ImGUI-панель управления.



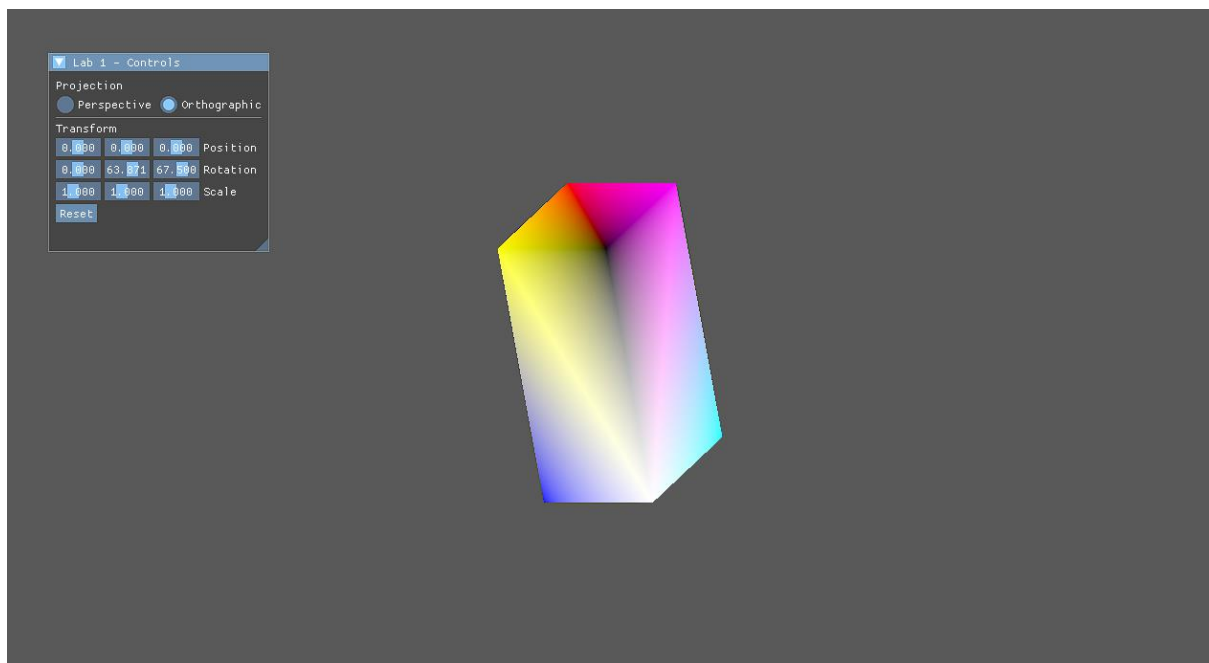


Рис. 2. Сравнение проекций: слева — перспективная (объект с перспективным искажением), справа — ортогографическая (размеры не зависят от расстояния до камеры).

Выводы

В ходе выполнения лабораторной работы я изучил основы трёхмерной графики и принципы работы с графическим API Vulkan. Освоил построение 3D-объектов из вершин и индексов, работу с MVP-матрицей (Model-View-Projection) для проецирования трёхмерной сцены на двумерный экран, а также разницу между перспективной и ортогографической проекциями.

Научился создавать и использовать основные объекты Vulkan: буферы для передачи данных на GPU, дескрипторы для связывания ресурсов с шейдерами, графический pipeline для описания конвейера отрисовки. Разобрался с вершинным и фрагментным шейдерами и их ролью в конвейере.

Также получил опыт интеграции ImGui для создания интерактивного интерфейса управления сценой.